

Lec 3

HEBB Network

Dr. Mohammad Mhawish

HEBB Network

- For a neural net, the Hebb learning rule is a simple one. Let us understand it. Donald Hebb stated in 1949 that in the brain, the learning is performed by the change in the synaptic gap.
- Hebb explained it: "When an axon of cell A is near enough to excite cell B, and repeatedly or permanently takes place in firing it, some growth process or metabolic change takes place in one or both the cells such that A's efficiency, as one of the cells firing B, is increased".
- According to the Hebb rule, the weight vector is found to increase proportionately to the product of the input and the learning signal.

HEBB Neural Network

- In Hebb learning, if two interconnected neurons are 'on' simultaneously then the weights associated with these neurons can be increased by the modification made in their synaptic gap (strength). The weight update in Hebb rule is given by

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad \longleftarrow \Delta w_i$$

- The Hebb rule is more **suited for bipolar data than binary data**. If binary data is used, the above weight updation formula cannot distinguish two conditions namely:
 1. A training pair in which an input unit is "on" and target value is "off".
 2. A training pair in which both the input unit and the target value are "off".
- **the Hebb rule results in better learning if the input / output both are in bipolar form.**
(-1/1)
- **Procedure** :Hebb-Learning requires only **one pass** through the training set.
- The activation function of input layer is **identity function**

- The training algorithm of Hebb network is given below:

Step 0: First initialize the weights. Basically in this network they may be set to zero, i.e., $w_i = 0$ for $i = 1$ to " n " where n may be the total number of input neurons.

Step 1: Steps 2–4 have to be performed for each input training vector and target output pair, $s:t$.

Step 2: Input units activations are set. Generally, the activation function of input layer is identity function:
 $x_i = s_i$ for $i = 1$ to n .

Step 3: Output units activations are set: $y = t$.

Step 4: Weight adjustments and bias adjustments are performed:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

$$b(\text{new}) = b(\text{old}) + y$$

The five steps complete the algorithmic process. In Step 4, the weight updation formula can also be given in vector form as

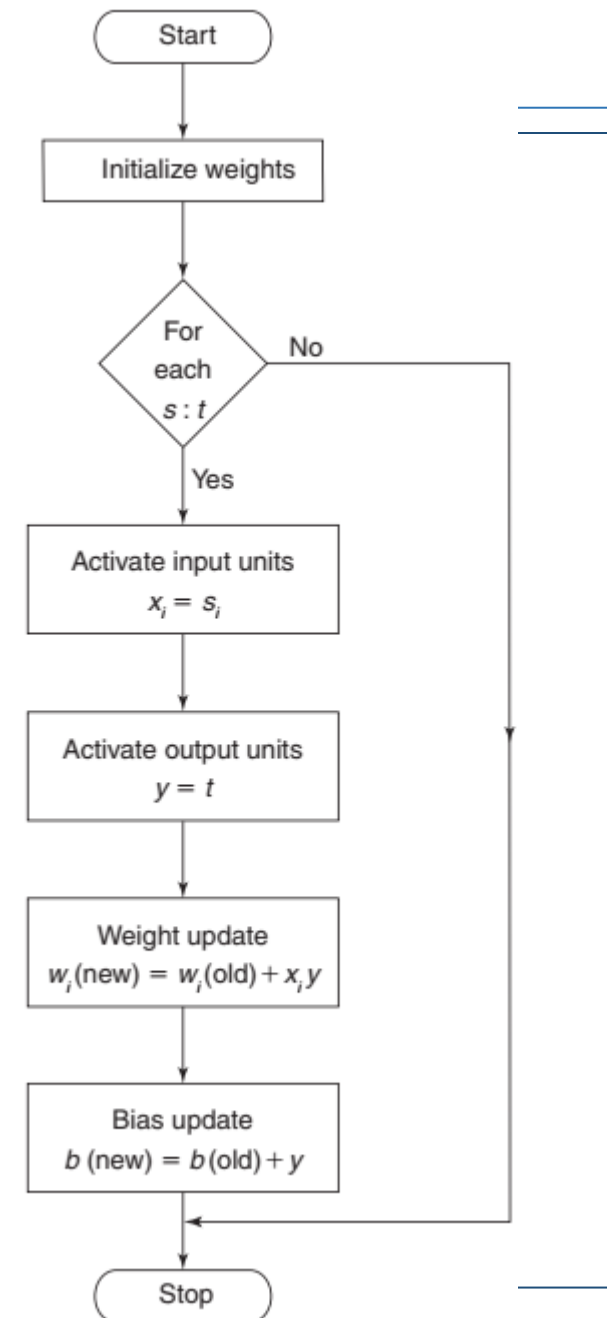
$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

Here the change in weight can be expressed as
 $\Delta w = xy$

As a result,

$$w(\text{new}) = w(\text{old}) + \Delta w$$

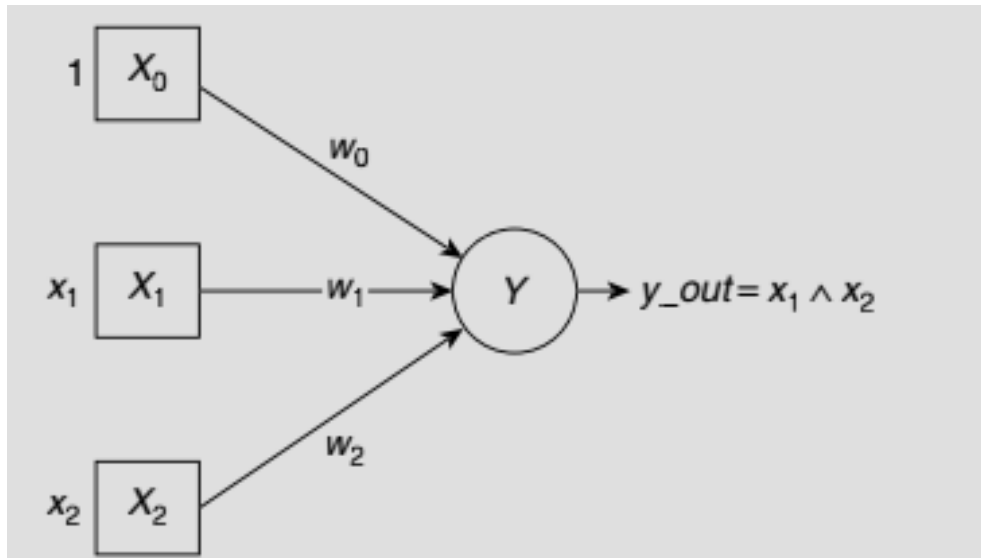
The Hebb rule can be used for pattern association, pattern categorization, pattern classification and over a range of other areas.



Try:

- **Realizing the logical AND function through Hebb learning**

To realize a two input *AND* function we need a net with two input units and one output unit. A bias is also needed. Hence the structure of the required neural net should be as shown in Fig. Moreover, the input and output signals must be in *bipolar* form, rather than the *binary* form, so that the net may be trained properly.



Input Patterns			Output
x_0	x_1	x_2	t
+1	1	1	1
+1	1	-1	-1
+1	-1	1	-1
+1	-1	-1	-1

- During the training process, all weights are initialized to 0. Therefore initially $w_0 = w_1 = w_2 = 0$.
- At each training instance, the weights are changed according to the formula $w_i(\text{new}) = w_i(\text{old}) + x_i y$

#	Training Pattern			Target output	Weight Adjustments			Weights		
	x_0	x_1	x_2	t	Δw_0	Δw_1	Δw_2	w_0	w_1	w_2
0								0	0	0
1	1	1	1	1	1	1	1	1	1	1
2	1	1	-1	-1	-1	-1	1	0	0	2
3	1	-1	1	-1	-1	1	-1	-1	1	1
4	1	-1	-1	-1	-1	1	1	-2	2	2

- **Step 1 :**

- Set weight and bias to zero, $w = [0 \ 0 \ 0]^T$ and $b = 0$.

- **Step 2 :**

- Set input vector $X_i = S_i$ for $i = 1$ to 4.

- $X_1 = [-1 \ -1 \ 1]^T$

- $X_2 = [-1 \ 1 \ 1]^T$

- $X_3 = [1 \ -1 \ 1]^T$

- $X_4 = [1 \ 1 \ 1]^T$

Step 3 :

Output value is set to $y = t$.

Step 4 :

Modifying weights using Hebbian Rule:

First iteration –

$$w(\text{new}) = w(\text{old}) + x_1 y_1 = [0 \ 0 \ 0]^T + [-1 \ -1 \ 1]^T \cdot [-1] = [1 \ 1 \ -1]^T$$

For the second iteration, the final weight of the first one will be used and so on.

Second iteration –

$$w(\text{new}) = [1 \ 1 \ -1]^T + [-1 \ 1 \ 1]^T \cdot [-1] = [2 \ 0 \ -2]^T$$

Third iteration –

$$w(\text{new}) = [2 \ 0 \ -2]^T + [1 \ -1 \ 1]^T \cdot [-1] = [1 \ 1 \ -3]^T$$

Fourth iteration –

$$w(\text{new}) = [1 \ 1 \ -3]^T + [1 \ 1 \ 1]^T \cdot [1] = [2 \ 2 \ -2]^T$$

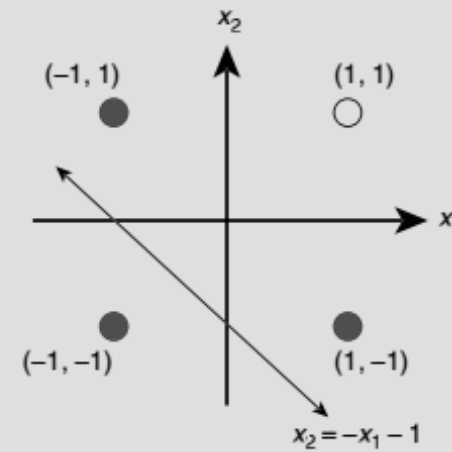
So, the final weight matrix is $[2 \ 2 \ -2]^T$

- <https://www.geogebra.org/geometry?lang=en>

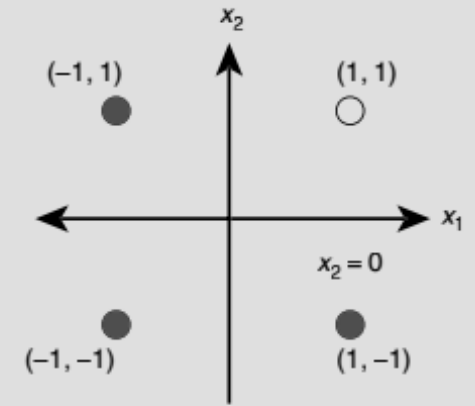
$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

we get

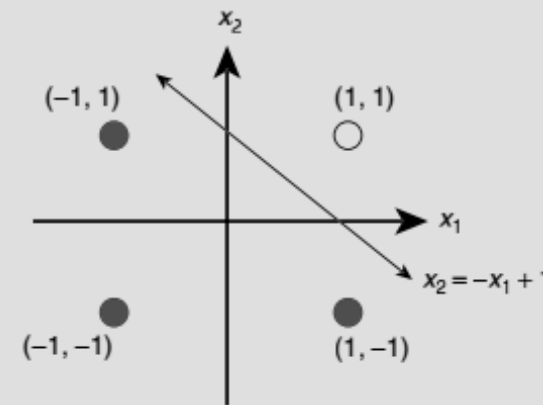
- (i) $x_2 = -x_1 - 1$, after the 1st training instance,
- (ii) $x_2 = 0$, after the 2nd training instance,
- (iii) $x_2 = -x_1 + 1$, after the 3rd training instance, and finally,
- (iv) $x_2 = -x_1 + 1$, after the 4th training instance.



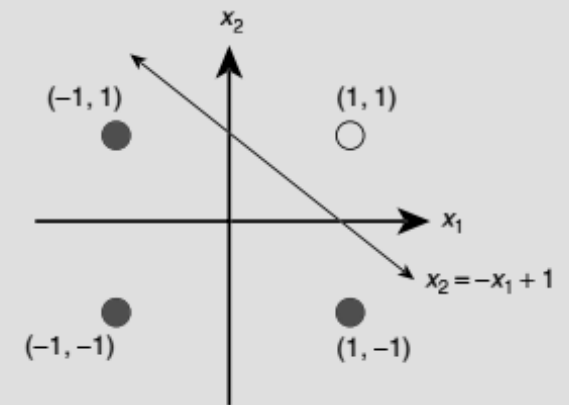
(a) After 1st pattern (1, 1)



(b) After 2nd pattern (1, -1)



(c) After 3rd pattern (-1, 1)



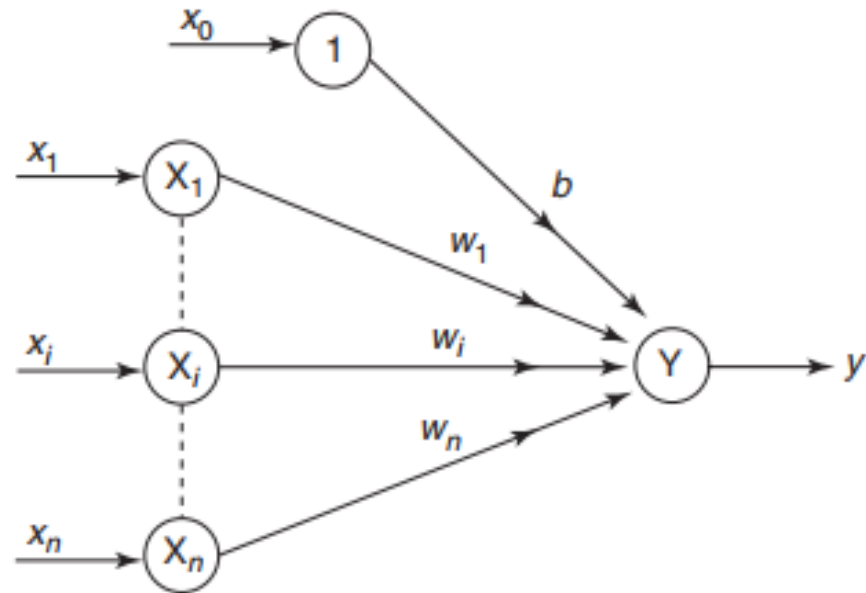
(d) After 4th pattern (-1, -1)

PERCEPTRON NETWORKS

- Perceptron networks come under single-layer feed-forward networks and are also called *simple perceptrons*.
- various types of perceptrons were designed by Rosenblatt (1962) and Minsky-Papert (1969, 1988). However, a simple perceptron network was discovered by Block in 1962.

Structure

- There are n input neurons, (1 or many output neuron) and a bias.
- The input-layer and output-layer neurons are connected through a directed communication link, which is associated with weights.
- The goal of the perceptron net is **to classify the input pattern as a member or not a member to a particular class.**



PERCEPTRON NETWORKS

The notable points regarding **Procedure** Perceptron-Learning are given below:

1. The **input** vectors are allowed to be either **binary or bipolar**. However, **the outputs must be in bipolar form**.
2. The **bias** w_0 is adjustable but the **threshold** used in the activation function is **fixed**.
3. Learning takes place only when the computed **output does not match** the **target**.

perceptron learning rule

- **The perceptron learning rule is explained as follows:**
- Consider a finite ***n*** number of input training vectors ***x***, with their associated target ***t*** (desired) values, $x(n)$ and $t(n)$, where "*n*" ranges from 1 to *N*.
- The **target** (*t*) is either +1 or -1.
- The **output** (*y*) is obtained on the basis of the **net input calculated** and **activation function being applied over the net input**.

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

The weight updating in case of perceptron learning is as shown:

- If $y \neq t$, then
 - $w \text{ (new)} = w \text{ (old)} + atx$ (a –learning rate)
 - else, we have
 - $w \text{ (new)} = w \text{ (old)}$
- The weights can be initialized at any values in this method.
- The perceptron rule convergence theorem states that “If there is a weight vector W , such that $f(x(n) W) = t(n)$, for all n , then for any starting vector w_1 , the perceptron learning rule will converge to a weight vector that gives the correct response for all training patterns, and this learning takes place within a finite number of steps provided that the solution exists.’

Perceptron Training Algorithm for Single Output Classes

Step 0: Initialize the weights and the bias (for easy calculation they can be set to zero). Also initialize the learning rate $\alpha (0 < \alpha \leq 1)$. For simplicity α is set to 1.

Step 1: Perform Steps 2–6 until the final stopping condition is false.

Step 2: Perform Steps 3–5 for each training pair indicated by $s:t$.

Step 3: The input layer containing input units is applied with identity activation functions:

$$x_i = s_i$$

Step 4: Calculate the output of the network. To do so, first obtain the net input:

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

where “ n ” is the number of input neurons in the input layer. Then apply activations over the net input calculated to obtain the output:

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Step 5: *Weight and bias adjustment:* Compare the value of the actual (calculated) output and desired (target) output.

If $y \neq t$, then

$$w_i(\text{new}) = w_i(\text{old}) + \alpha tx_i$$

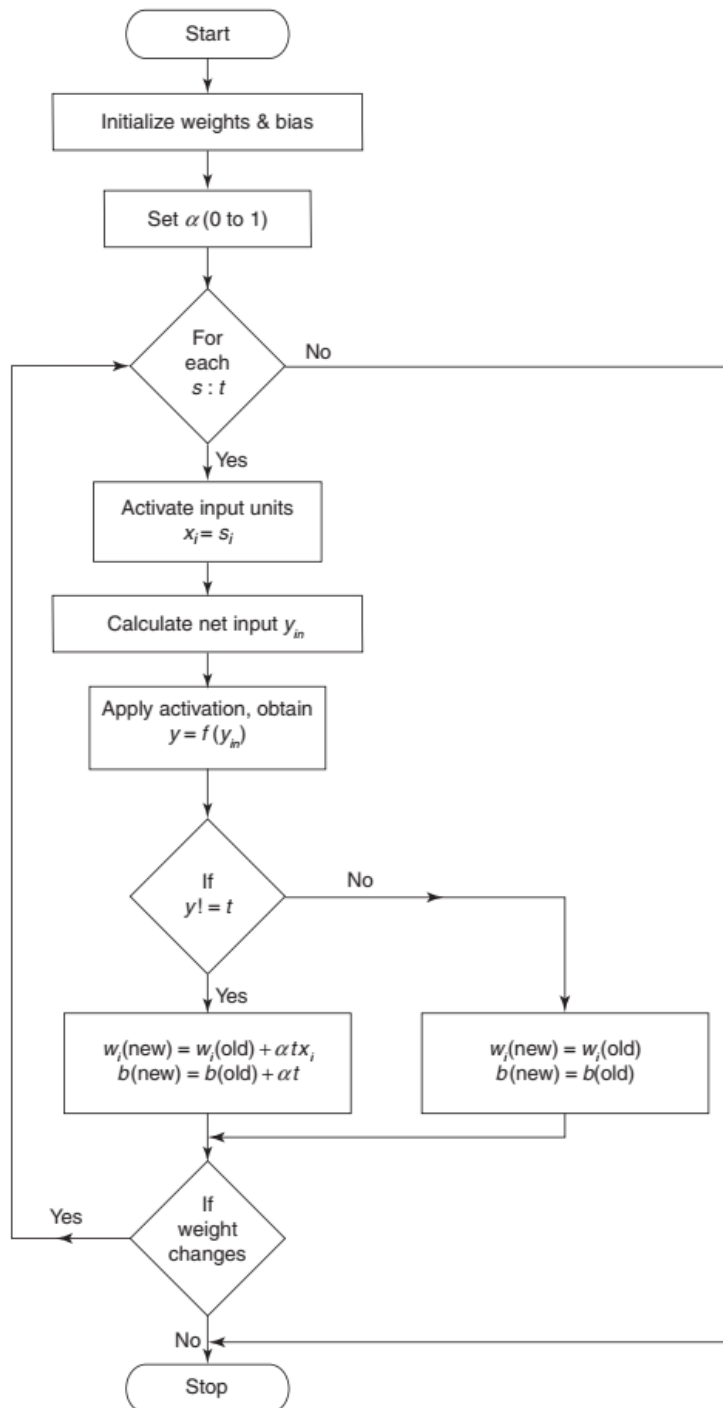
$$b(\text{new}) = b(\text{old}) + \alpha t$$

else we have

$$w_i(\text{new}) = w_i(\text{old})$$

$$b(\text{new}) = b(\text{old})$$

Step 6: Train the network until there is no weight change. This is the stopping condition for the network. If this condition is not met, then start again from Step 2.



- Try : *Learning the logical AND function by a perceptron*
suppose $w_0=w_1=w_2= 0$ and learning rate =1